



INF1040
Projeto de Programação Modular

Grupo 03 - ULL Exchange Core

2510071 Antônio Vita Fonseca

2510619 Gabriel Félix de Farias Bereicoa

2410804 Guilherme Machado de Bragança Soares

2410425 Lucas de Menezes Baptista

Professor:

Flavio Heleno Bevilacqua e
Silva

Rio de Janeiro
Maio 2026

OBJETIVO DA APLICAÇÃO

O objetivo principal desta aplicação é o desenvolvimento de um Motor de Matching de Alta Performance (Matching Engine) para simulação de um mercado de capitais simplificado. Dessa forma, o sistema visa processar ordens de compra e venda de ativos financeiros com foco em baixa latência (Ultra-Low Latency - ULL) e integridade de dados.

Além disso, a aplicação busca demonstrar a viabilidade técnica da integração entre uma linguagem de alto nível (Python), utilizada para orquestração, geração de dados e análise de resultados, e uma linguagem de baixo nível (C), responsável pelo processamento crítico de dados em memória.

Mais especificamente, objetiva-se implementar um Livro de Ofertas (Order Book) que organize e processe ordens seguindo a prioridade de Preço-Tempo; garantir que o processamento massivo de dados ocorra sem vazamentos de memória e com isolamento estrito entre os módulos de validação, execução e persistência; registrar o histórico de negociações em formato binário de alta densidade (Ledger), assegurando a rastreabilidade total das transações realizadas; eliminar gargalos de I/O de disco durante o ciclo crítico de execução através do pré-carregamento de dados (Data Ingestion) e uso de buffers contíguos em RAM.

1. ESPECIFICAÇÃO DE REQUISITOS E RESTRIÇÕES

A especificação de requisitos e restrições constitui o alicerce técnico do projeto, definindo de forma rigorosa as funcionalidades, as qualidades de desempenho e as limitações arquitetônicas que o sistema deve obedecer. Este tópico atua como um contrato de engenharia entre os desenvolvedores, garantindo que o motor de matching atinja as metas de baixa latência e determinismo exigidas pelo mercado financeiro. Enquanto os requisitos detalham as ações lógicas e os fluxos de dados, as restrições impõem limites tecnológicos — como a escolha de linguagens e paradigmas de programação — assegurando que a integridade e a segurança do ecossistema sejam mantidas durante todo o ciclo de desenvolvimento.

1.1. Requisitos Funcionais

Os Requisitos Funcionais descrevem as transformações que o sistema deve realizar sobre os dados de entrada para produzir os resultados esperados. Eles definem o comportamento lógico da aplicação em resposta a diferentes estímulos do mercado simulado.

1.1.1 Geração de Massa de Dados

O sistema deve possuir um script capaz de gerar arquivos CSV simulando o mercado, com parâmetros ajustáveis de volume de ordens e percentual de falhas.

1.1.2 Carga de Dados em Memória (Ingestion)

O motor deve ler o arquivo CSV e carregar todas as ordens em um buffer contíguo de memória (array de structs) antes de iniciar o cronômetro de performance.

1.1.3 Validação de Consistência

O sistema deve invalidar ordens que apresentem preços ou quantidades menores ou iguais a zero, marcando-as para que não participem do matching.

1.1.4 Execução por Prioridade Preço-Tempo

O motor de matching deve cruzar ordens priorizando o melhor preço disponível e, em caso de empate, a ordem que foi carregada primeiro (FIFO).

1.1.5 Gestão de Execuções Parciais

Se uma ordem agressora tiver quantidade superior à melhor oferta do livro, o sistema deve executar a parte disponível e manter o restante como uma nova oferta no Order Book.

1.1.6 Registro de Auditoria (Ledger)

Para cada negócio realizado (match), o sistema deve gravar uma transação em um arquivo binário contendo os IDs das ordens, preço e quantidade executada.

1.1.7 Não processamento de ordens inválidas

O motor de matching não deve, sob nenhuma circunstância, tentar cruzar ordens que tenham sido marcadas como inválidas pelo módulo Validator.

1.1.8 Não persistência de ordens abertas no Ledger

O Ledger não deve registrar ordens que foram apenas inseridas no Book; apenas execuções confirmadas (trades) devem ser persistidas.

1.1.9 Não bloqueio por Interface

O processamento do motor em C não deve ser interrompido ou atrasado por chamadas de interface do Python durante o loop de matching.

1.1.10 Não alteração de ordens executadas

Uma vez que uma transação é registrada no Ledger, os dados daquela transação não devem ser alterados ou deletados pelo sistema.

1.1.11 Não cruzamento de ordens do mesmo cliente

O sistema não deve permitir que um cliente compre dele mesmo (Self-Match), caso essa regra seja definida na lógica de negócio.

1.1.12 Ordenamento real do mercado

O arquivo CSV gerado precisa estar em ordem crescente de horário para simular corretamente o mercado real.

1.1.13 Sincronização entre Estado do Livro e Registro de Transações

O sistema deve garantir a atomicidade lógica entre a remoção/atualização de uma ordem no Book e o seu registro no Ledger. Nenhuma ordem pode ser removida do Livro de Ofertas por motivo de execução sem que uma transação equivalente tenha sido despachada para o módulo de persistência, evitando discrepâncias entre o inventário de ordens e o histórico de trades.

1.2. Requisitos Não-Funcionais

Os Requisitos Não-Funcionais estabelecem os critérios de qualidade, desempenho e restrições de arquitetura que delimitam como as funcionalidades serão implementadas.

1.2.1 Baixa Latência (ULL)

O motor de matching deve ser implementado em C puro, evitando alocações dinâmicas de memória (malloc) durante o loop crítico de cruzamento de ordens.

1.2.2 Portabilidade da Biblioteca

O núcleo em C deve ser compilado como uma Shared Library (.so ou .dll), permitindo que seja invocado por diferentes interfaces (Python, C++ ou Java).

1.2.3 Eficiência de I/O

A gravação no Ledger deve ser feita em formato binário para minimizar o overhead de escrita em disco e garantir que o processo de persistência não gargale o motor de matching.

1.2.4 Gerenciamento de Memória

O sistema deve garantir a liberação de todos os recursos alocados ao final da execução, não apresentando memory leaks (vazamentos de memória).

1.3. Restrições

As restrições de projeto estabelecem limitações impostas ao desenvolvimento que não dependem da vontade da equipe técnica, mas de diretrizes pedagógicas, arquiteturais ou de infraestrutura. Diferente dos requisitos funcionais, as restrições delimitam o espaço de solução, garantindo que a implementação siga padrões específicos de linguagem, estilo de programação e segurança, assegurando a compatibilidade entre os diferentes módulos do ecossistema.

1.3.1 Restrição de Linguagens

O código do sistema deve ser escrito estritamente nas linguagens C ou Python.

1.3.2 Paradigma de Programação

É vedada a utilização de Programação Orientada a Objetos (classes) no núcleo em C, priorizando o paradigma imperativo/procedural para minimizar o overhead de memória e garantir o determinismo.

1.3.3 Restrição de Dados

O código do sistema não pode utilizar Bancos de Dados ou qualquer outra forma de persistência de dados.

2. DIAGRAMA DE ARQUITETURA DE MÓDULOS

A arquitetura do sistema foi desenhada seguindo o paradigma de Programação Modular, com uma clara separação de responsabilidades e uma hierarquia de controle centrada no motor de execução. A estrutura utiliza um modelo de comunicação Cliente-Servidor Interno, onde o módulo de Matching atua como o orquestrador (cliente) que consome serviços especializados dos demais módulos (servidores).

MÓDULOS

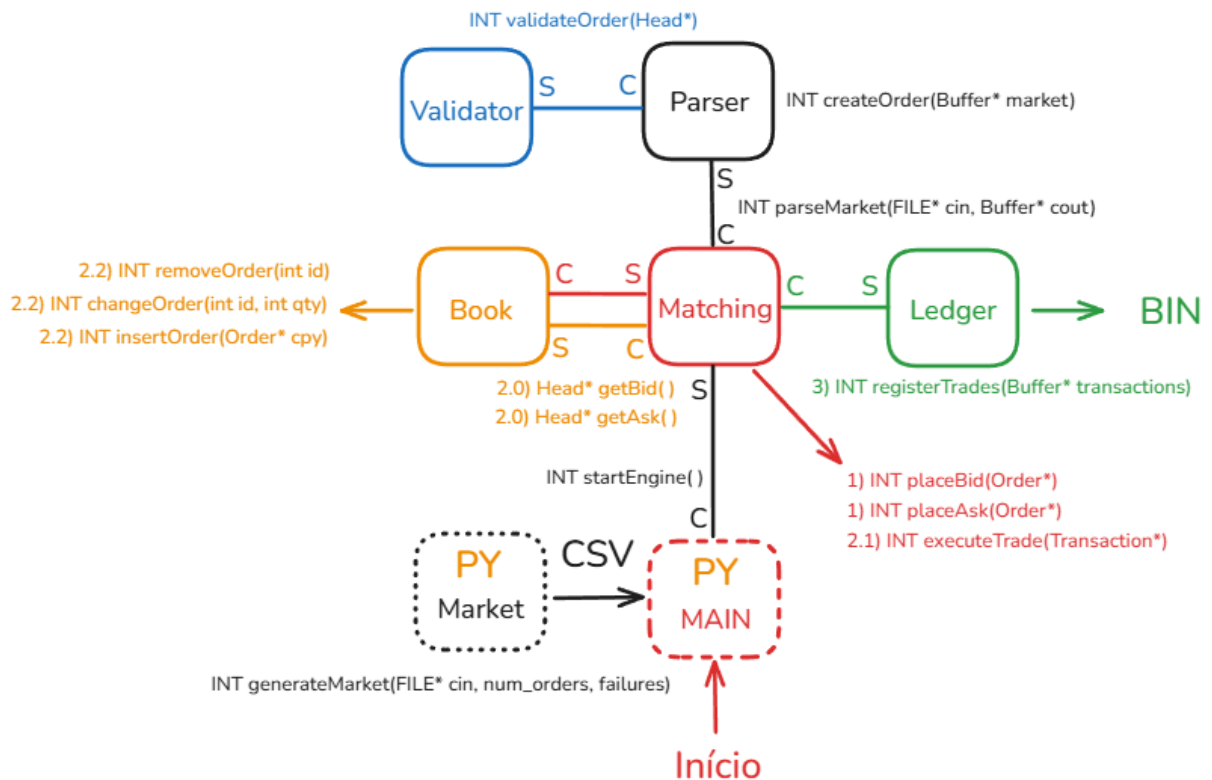


Figura 2 - Diagrama de Blocos da Arquitetura.

Fonte: Autoral, 2026

2.1. Hierarquia e Fluxo de Controle

A execução é dividida em duas camadas principais: a camada de orquestração (Python), responsável pelo setup do ambiente, geração da massa de dados via CSV e invocação da biblioteca dinâmica em C. Esta camada garante a flexibilidade na interface com o usuário e na análise posterior de resultados. E a camada de execução crítica (C), em que, uma vez iniciado o motor, o controle é transferido integralmente para o núcleo em C. O fluxo segue uma sequência lógica de ingestão, validação, cruzamento e persistência, minimizando o tempo de resposta através do uso de memória RAM contígua.

2.2. Fluxo de Execução do Programa

O funcionamento da aplicação segue um fluxo sequencial e hierárquico, garantindo que o processamento de alta performance ocorra sobre dados previamente tratados e validados. O ciclo de vida do programa é dividido nas seguintes etapas:

2.2.1 Inicialização e Setup (Main)

O processo inicia-se na camada Python (Main), onde é realizada a preparação do ambiente da seguinte forma:

`generateMarket()`: Invocação do script responsável por gerar o cenário de mercado (arquivo CSV).

`startEngine()`: Chamada da função principal no núcleo em C, transferindo o controle para o módulo de Matching.

2.2.2 Preparação de Dados (Matching & Parse)

Uma vez no ambiente C, o motor prepara a memória para o processamento:

`parseMarket()`: O Matching solicita ao módulo Parse a leitura dos dados brutos.

`createOrders()`: O Parse realiza a alocação do buffer e a conversão das strings do CSV para estruturas binárias na memória RAM.

`validateOrder()`: Antes de retornar o controle ao motor, o Parse invoca o Validator para assegurar a integridade das ordens.

2.2.3 Ciclo Crítico de Negociação (Matching & Book)

Com o buffer pronto, o Matching inicia o loop de cruzamento:

`placeBid / placeAsk()`: O motor recebe cada ordem e decide o fluxo de processamento com base no lado da operação.

`getAsk / getBid()`: O Matching consulta o Book para verificar a existência de contrapartes no topo do livro que satisfaçam as condições de preço.

`remove / change / insert()`: Dependendo do resultado do cruzamento, o motor comanda o Book para remover ordens executadas, alterar quantidades em matches parciais ou inserir novas ordens pendentes.

2.2.4 Persistência e Auditoria (Ledger)

Ao finalizar todos os cruzamentos possíveis com sucesso:

`registerTrades()`: O Matching envia todas as transações (guardadas na memória local) por meio de uma cabeça para o Ledger, que realiza a gravação em formato binário para garantir o registro histórico imutável da operação.

3. COMPONENTES DO SISTEMA E DIVISÃO DE RESPONSABILIDADES

O sistema é composto por sete módulos interdependentes, cada um projetado para executar uma tarefa específica dentro do ciclo de vida de uma ordem financeira. A arquitetura foi planejada para permitir o desenvolvimento paralelo, onde cada integrante do grupo assumiu a responsabilidade técnica por módulos específicos, atuando como o "desenvolvedor líder" daquela funcionalidade e de seus respectivos programas de teste.

3.1. Lucas

Responsável pela parte da programação em Python, que é o início da aplicação e geração do CSV, além de cuidar do módulo final, que registra os resultados em binário.

3.1.1 Market & Main (Python)

Módulos responsáveis pela interface externa. O Market atua na geração da massa de dados sintética, enquanto a Main provê o ponto de entrada da aplicação, realizando a carga da biblioteca dinâmica em C e a orquestração do experimento.

3.1.2 Ledger (C)

Módulo de persistência e auditoria. Grava de forma imutável e binária cada transação realizada, permitindo a verificação posterior dos resultados da simulação.

3.2. Gabriel

Responsável pela transposição das linhas do arquivo CSV em um array de estruturas das ordens, além de validar cada uma delas para garantir a integridade do sistema e evitar possíveis erros nos módulos seguintes.

3.2.1 Parser & Validator (C)

Responsáveis pela camada de ingestão. O Parser realiza a leitura eficiente do arquivo CSV e a conversão para binário em RAM; o Validator atua na higienização desses dados, garantindo que apenas ordens logicamente consistentes cheguem ao motor.

3.3. Antônio

Responsável pela engine do sistema. Este módulo que orquestra e chama os demais conforme o necessário, além de realizar as operações do mercado.

3.3.1 Matching Engine (C)

O núcleo decisório do sistema. Implementa a lógica de cruzamento de ofertas, consumindo os dados do Parser e coordenando as ações entre o Book e o Ledger.

3.4. Guilherme

Responsável pela organização das ordens em uma Heap para Bids e outra para Vendas, garantindo agilidade e a prioridade das ordens.

3.4.1 Order Book (C)

Módulo de gestão de estado, responsável por manter as filas de compra e venda organizadas por prioridade. É o componente encarregado de gerenciar a memória das ordens que aguardam contraparte no mercado.

4. CASOS DE TESTE

A seção de testes define os critérios de aceitação e os mecanismos de controle de qualidade para garantir que cada componente do sistema opere conforme o planejado. O uso de casos de teste automatizados é fundamental em um ambiente de alta performance para assegurar o determinismo do motor, garantindo que o processamento de grandes volumes de dados não resulte em comportamentos imprevistos ou falhas de segmentação. Através da definição rigorosa de padrões de retorno e da simulação de cenários de erro, estabelece-se uma base confiável para o desenvolvimento orientado por testes (TDD), facilitando a identificação precoce de bugs e garantindo a integridade da comunicação entre os módulos em Python e C.

4.1. Padrões de Retorno

Código	Classificação	Descrição Técnica
2	Sucesso com Observação	Match Parcial: A ordem foi executada parcialmente e o saldo permanece no Book.
1	Sucesso com Observação	Match Total: A ordem foi executada integralmente e removida do Book.
0	Sucesso Absoluto	Operação concluída conforme esperado (ex: inserção, inicialização ou gravação).
-1	Erro de Negócio	Ordem Inválida: Falha na validação de consistência (preço, quantidade ou lado).
-2	Erro de Referência	ID não encontrado: Tentativa de alteração ou remoção de ordem inexistente no Book.
-3	Erro de Recurso	Falha Crítica: Erro de alocação de memória (malloc) ou erro de I/O no disco.

-4	Erro de Geração de Arquivo	Falha na Geração de Documento: Ocorreu algum erro na criação de um arquivo.
-5	Erro de Fluxo	Falha na Transação: O cruzamento ocorreu, mas houve erro ao despachar para o Ledger.
-6	Erro de Lógica	Alteração Inválida: Tentativa de modificar uma ordem para uma quantidade impossível.
-7	Falha na Transição das Linguagens	Falha na Chamada: Erro ao tentar chamar a interface em C por meio da interface em Python..

Tabela 4.1 - Códigos de Retorno das Funções

Fonte: Autoral, 2026

4.2. Testes da Main

Função	Retorno (Sucesso)	Retorno (Erro)	Casos de Teste (Válidas/Inválidas)
generateMarket (FILE* cin, num_orders, failures)	0 (Sucesso)	-4 (Falha na geração do mercado)	Gerar 100 ordens mistas (compras, vendas, parciais e critérios de desempate) em ordem crescente.
startEngine()	0 (Sucesso)	-7 (Falha na comunicação das linguagens)	Iniciar o motor e garantir a transferência de controle para o C.

Tabela 4.2 - Testes do Módulo Main

Fonte: Autoral, 2026

4.3. Testes do Parser

Função	Retorno (Sucesso)	Retorno (Erro)	Casos de Teste (Válidas/Inválidas)
createOrders(FILE* market, int* count)	0 (Sucesso)	-1 (Erro na criação da Ordem)	Separar e carregar as 100 ordens geradas na RAM corretamente.
freeBuffer(Order* buffer)	Sem Retorno (Função Embutida)	Sem Retorno (Função Embutida)	Garantir a limpeza total da memória após o fim do processamento.

Tabela 4.3 - Testes do Módulo Parser

Fonte: Autoral, 2026

4.4. Testes do Validator

Função	Retorno (Sucesso)	Comportamento em Erro	Casos de Teste (Válidas/Inválidas)
validateOrder (Head*)	0 (Ordem Válida)	-1 (Ordem Inválida), -2 (Falha na Validação)	Testar ordens com preço ≤ 0 , quantidade ≤ 0 ou lado diferente de 'C'/'V'.

Tabela 4.4 - Testes do Módulo Validator

Fonte: Autoral, 2026

4.5. Testes do Matching

Função	Retorno (Sucesso)	Retorno (Erro)	Casos de Teste (Válidas/Inválidas)
placeAsk(Order* incoming)	0 (Sucesso)	-1 (Ordem inválida)	Inserção de vendas no Book quando não há compradores.

placeBid(Order* incoming)	0 (Sucesso)	-1 (Ordem inválida)	Inserção de compras no Book quando não há vendedores.
executeTrade(Transaction*)	0 (Proc.), 1 (Total), 2 (Parcial)	-5 (Falha na transação)	Matches totais, parciais, desempate por tempo (FIFO).
parseMarket(FILE* cin, Buffer* cout)	0 (Sucesso)	-3 (Falha na geração do mercado)	Validação do fluxo completo de leitura e início do motor.

Tabela 4.5 - Testes do Módulo Matching

Fonte: Autoral, 2026

4.6. Testes do Booking

Função	Retorno (Sucesso)	Retorno (Erro)	Casos de Teste (Válidas/Inválidas)
getBid() / getAsk()	Ponteiro para o Cabeça do heap	NULL (Fila vazia)	Verificar se o topo é sempre a melhor oferta de preço.
insertOrder(Order* cpy)	0 (Sucesso)	-3 (Erro de memória)	Organização da fila (Heap/Lista) e manutenção da ordem temporal.
removeOrder(int id)	0 (Sucesso)	-2 (ID não encontrado)	Remoção de ordens após execução total ou cancelamento.
changeOrder(int id, int qty)	0 (Sucesso)	-2 (ID não encontrado), -6 (Quantidade inválida)	Atualização de quantidade em casos de match parcial.

Tabela 4.6 - Testes do Módulo Book

Fonte: Autoral, 2026

4.7. Testes do Ledger

Função	Retorno (Sucesso)	Retorno (Erro)	Casos de Teste (Válidas/Inválidas)
registerTrades (Buffer* transactions)	0 (Sucesso)	-3 (Falha de escrita / Disco cheio)	Salvar 100 transações completas e verificar integridade do binário.

Tabela 4.7 - Testes do Módulo Ledger

Fonte: Autoral, 2026

4.8. Casos de Teste Integrado

Cenário	Entrada	Resultado Esperado	Módulos Envolvidos	Código Esperado
Fluxo completo sem erros	CSV com 100 ordens válidas	Todas as ordens processadas, matches executados corretamente e ledger preenchido	Main, Parser, Validator, Matching, Book, Ledger	0
Presença de ordens inválidas	CSV com ordens com preço ≤ 0 ou quantidade ≤ 0	Ordens inválidas ignoradas, nenhuma participação no matching	Parser, Validator, Matching	-1
Match total simples	1 ordem de compra = 1 ordem de venda (mesmo preço / quantidade)	Ambas removidas do book, 1 transação registrada	Matching, Book, Ledger	1
Match parcial	Compra maior que venda disponível	Venda removida, compra permanece	Matching, Book, Ledger	2

		parcialmente no book		
Prioridade preço-tempo (FIFO)	Múltiplas ordens com mesmo preço	Ordem mais antiga executada primeiro	Matching, Book	1
Sem contraparte	Apenas ordens de compra (ou venda)	Ordens inseridas no book sem execução	Matching, Book	0
Falha no Ledger	Simulação de erro de escrita (disco cheio)	Matching detecta falha e retorna erro sem inconsistência no book	Matching, Ledger	-5
Sincronização Book-Ledger	Execução normal	Toda ordem removida do book possui transação correspondente no ledger	Matching, Book, Ledger	0
Self-match proibido	Ordens do mesmo cliente	Nenhuma execução realizada entre elas	Matching	-1
CSV fora de ordem temporal	Ordens fora de ordem crescente de tempo	Execução incorreta detectável (ou rejeição na validação, dependendo da implementação)	Parser, Validator, Matching	-1
Volume alto (stress test)	CSV com grande volume (ex: 1 milhão de ordens)	Execução completa sem falhas e sem degradação crítica	Todos	0
Liberação de memória	Execução completa seguida de finalização	Nenhum memory leak detectado	Parser, Book, Matching	0

Alteração inválida após match parcial	Tentativa de alterar quantidade inconsistente	Operação rejeitada	Book	-6
Ledger consistente	Execução com múltiplos trades	Arquivo binário íntegro e rastreável	Ledger	0

Tabela 4.8 - Tabela dos Testes Integrados

Fonte: Autoral, 2026